Practice Questions- Collections

Java Beginner

Aug 24, 2026

Java Collections – Practice Coding Set

Concept Progression

Q	MAIN CONCEPT	COLLECTION	DIFFICULTY
1	Add elements conditionally	ArrayList	Easy
2	Search and count occurrences	ArrayList	Easy
3	Filter elements	ArrayList	Easy
4	Remove selected elements	ArrayList	Easy
5	Update collection with additions/removals	ArrayList	Easy–Medium
6	Process ordered data	LinkedList	Easy
7	Find elements greater than previous	LinkedList	Easy
8	Identify repeated elements	LinkedList	Easy–Medium
9	Remove duplicates using HashSet	HashSet	Easy
10	Find common elements using HashSet	HashSet	Easy
11	Find second distinct minimum	HashSet + sorting	Medium
12	Count frequencies using HashMap	HashMap	Medium
13	Find first/last unique character	HashMap	Medium
14	Preserve insertion order with unique data	LinkedHashSet	Medium
15	Combine List + Set + Map concepts	Collections	Medium

The progression deliberately moves from straightforward collection operations to duplicate handling, uniqueness, frequency counting, and combined collection logic.

Question 1 — Add New Product Codes

Problem Statement

An online store maintains an `ArrayList<Integer>` containing the product codes currently available.
A second array contains newly received product codes.
For every new code:

- add it if it is not already present;
- ignore it if the code already exists.

Return the updated `ArrayList`.

Required Method

```
public static ArrayList<Integer> updateProducts(
    ArrayList<Integer> products,
    int[] newProducts)
```

Approach

Traverse the `newProducts` array one element at a time.
For every product code:

```
if (!products.contains(code))
```

add it to the list.
This maintains the original order and prevents duplicates.

Full Java Running Code

```java
import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static ArrayList<Integer> updateProducts(
            ArrayList<Integer> products,
            int[] newProducts) {

        for (int code : newProducts) {
            if (!products.contains(code)) {
                products.add(code);
            }
        }

        return products;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        ArrayList<Integer> products = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            products.add(sc.nextInt());
        }

        int m = sc.nextInt();
        int[] newProducts = new int[m];

        for (int i = 0; i < m; i++) {
            newProducts[i] = sc.nextInt();
        }

        ArrayList<Integer> result =
                updateProducts(products, newProducts);

        for (int value : result) {
            System.out.print(value + " ");
        }

        sc.close();
    }
}
```

Explanation

The main logic is:

```java
if (!products.contains(code)) {
    products.add(code);
}
```

`contains()` checks whether the value already exists. Only new values are appended, so duplicates are avoided while the existing order is preserved.

Question 2 — Count a Requested Item

Problem Statement

A warehouse stores item numbers in an `ArrayList<Integer>`.
Given an item number to search for:

- print how many times it occurs;
- if it does not appear at all, print `-1`.

Process multiple test cases.

Approach

For each test case:

1. Read the list.
2. Read the target item.
3. Traverse the `ArrayList`.
4. Increase a counter whenever the target is found.
5. Print the count or `-1`.

Full Java Running Code

```java
import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static int countItem(
            ArrayList<Integer> items,
            int target) {

        int count = 0;

        for (int value : items) {
            if (value == target) {
                count++;
            }
        }

        return count == 0 ? -1 : count;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();

        while (t-- > 0) {

            int n = sc.nextInt();

            ArrayList<Integer> items = new ArrayList<>();

            for (int i = 0; i < n; i++) {
                items.add(sc.nextInt());
            }

            int target = sc.nextInt();

            System.out.println(countItem(items, target));
        }

        sc.close();
    }
}
```

Explanation

The program maintains a `count` variable.
Whenever:

```
value == target
```

the counter increases.
If the final count remains zero, `-1` is returned.

Question 3 — Select High-Priority Tasks

Problem Statement

A project management system stores task priorities in an `ArrayList<Integer>`.
A task is considered high priority when its priority value is at least `7`.
Create and return a new `ArrayList` containing only those high-priority values.
Maintain their original order.

Approach

Traverse the original list and add values satisfying:

```
priority >= 7
```

to a new list.
Do not modify the original collection.

Full Java Running Code

```java
import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static ArrayList<Integer> findHighPriority(
            ArrayList<Integer> priorities) {

        ArrayList<Integer> result = new ArrayList<>();

        for (int priority : priorities) {
            if (priority >= 7) {
                result.add(priority);
            }
        }

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        ArrayList<Integer> priorities = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            priorities.add(sc.nextInt());
        }

        ArrayList<Integer> result =
                findHighPriority(priorities);

        for (int value : result) {
            System.out.print(value + " ");
        }

        sc.close();
    }
}
```

Explanation

A second `ArrayList` is created for the answer.
Each element is examined once. Values satisfying `priority >= 7` are added to the result, so the original ordering is automatically retained.

Question 4 — Remove Cancelled IDs

Problem Statement

A conference maintains registered participant IDs in an `ArrayList<Integer>`.
Some participants cancel their registration.
Given an array of cancelled IDs, remove every cancelled ID that currently exists in the list.
Return the updated collection.

Approach

Traverse the cancellation array.
For every ID, use:

```
participants.remove(Integer.valueOf(id));
```

Using `Integer.valueOf()` ensures that the value is removed rather than treating the ID as an array index.

Full Java Running Code

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static ArrayList<Integer> removeCancelled(
            ArrayList<Integer> participants,
            int[] cancelled) {

        for (int id : cancelled) {
            participants.remove(Integer.valueOf(id));
        }

        return participants;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        ArrayList<Integer> participants = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            participants.add(sc.nextInt());
        }

        int c = sc.nextInt();

        int[] cancelled = new int[c];

        for (int i = 0; i < c; i++) {
            cancelled[i] = sc.nextInt();
        }

        ArrayList<Integer> result =
                removeCancelled(participants, cancelled);

        for (int id : result) {
            System.out.print(id + " ");
        }

        sc.close();
    }
}
```

Explanation

The important statement is:

```
participants.remove(Integer.valueOf(id));
```

It removes the matching value from the `ArrayList`.
If the ID is not present, nothing is removed.

Question 5 — Maintain an Updated Queue

Problem Statement

A training institute keeps a list of enrolled student IDs.
During the day:

- some students withdraw;
- new students request admission.

For withdrawals, remove the ID if it exists.
For new admissions, add the ID only when it is not already present.
Return the final `ArrayList`.

Approach

Perform the two operations in order:

1. Remove all withdrawn IDs.
2. Process new IDs and add only those that are absent.

Full Java Running Code

```java
import java.util.ArrayList;
import java.util.Scanner;

public class Main {

    public static ArrayList<Integer> updateStudents(
            ArrayList<Integer> students,
            int[] withdrawn,
            int[] newStudents) {

        for (int id : withdrawn) {
            students.remove(Integer.valueOf(id));
        }

        for (int id : newStudents) {
            if (!students.contains(id)) {
                students.add(id);
            }
        }

        return students;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        ArrayList<Integer> students = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            students.add(sc.nextInt());
        }

        int w = sc.nextInt();
        int[] withdrawn = new int[w];

        for (int i = 0; i < w; i++) {
            withdrawn[i] = sc.nextInt();
        }

        int m = sc.nextInt();
        int[] newStudents = new int[m];

        for (int i = 0; i < m; i++) {
            newStudents[i] = sc.nextInt();
        }

        ArrayList<Integer> result =
                updateStudents(
                        students,
                        withdrawn,
                        newStudents);

        for (int id : result) {
            System.out.print(id + " ");
        }

        sc.close();
    }
}
```

Explanation

The method performs two stages.
First:

```
students.remove(Integer.valueOf(id));
```

removes withdrawn students.
Then:

```
if (!students.contains(id)) {
    students.add(id);
}
```

prevents duplicate registrations.

Question 6 — Detect Increasing Temperatures

Problem Statement

A weather station stores daily temperatures in a `LinkedList<Integer>`.
Create a new `LinkedList` containing every temperature that is greater than the temperature recorded immediately before it.
The first temperature should not be included.
Preserve the original order.

Approach

Start from index `1`.
Compare:

```
current temperature > previous temperature
```

If true, add the current temperature to the result.

Full Java Running Code

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {

    public static LinkedList<Integer> findIncreases(
            LinkedList<Integer> temperatures) {

        LinkedList<Integer> result = new LinkedList<>();

        for (int i = 1; i < temperatures.size(); i++) {

            if (temperatures.get(i) >
                    temperatures.get(i - 1)) {

                result.add(temperatures.get(i));
            }
        }

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        LinkedList<Integer> temperatures =
                new LinkedList<>();

        for (int i = 0; i < n; i++) {
            temperatures.add(sc.nextInt());
        }

        LinkedList<Integer> result =
                findIncreases(temperatures);

        for (int value : result) {
            System.out.print(value + " ");
        }

        sc.close();
    }
}
```

Explanation

The first element has no previous value, so processing starts from index `1`.
For every later element:

```
temperatures.get(i) > temperatures.get(i - 1)
```

checks whether the value increased.
This same ordered-neighbour comparison pattern is central to the `LinkedList` trend-analysis problems.

Question 7 — Find Increasing Scores

Problem Statement

A gaming application records player scores after each round in a `LinkedList<Integer>`.
Return a new list containing the scores that are strictly greater than the score from the previous round.
If there are no such scores, return an empty list.

Approach

Traverse the list from the second element.
Whenever the current score is greater than the previous score, add it to the answer.

Full Java Running Code

```
import java.util.LinkedList;
import java.util.Scanner;

public class Main {

    public static LinkedList<Integer> findImprovedScores(
            LinkedList<Integer> scores) {

        LinkedList<Integer> result = new LinkedList<>();

        for (int i = 1; i < scores.size(); i++) {

            int current = scores.get(i);
            int previous = scores.get(i - 1);

            if (current > previous) {
                result.add(current);
            }
        }

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        LinkedList<Integer> scores = new LinkedList<>();

        for (int i = 0; i < n; i++) {
            scores.add(sc.nextInt());
        }

        LinkedList<Integer> result =
                findImprovedScores(scores);

        for (int score : result) {
            System.out.print(score + " ");
        }

        sc.close();
    }
}
```

Explanation

Each score is compared with the immediately preceding score.
Only:

```
current > previous
```

values are added.
The result therefore contains only the points where the score improved.

Question 8 — Find Repeated Visit IDs

Problem Statement

A mobile application stores page IDs visited by a user in a `LinkedList<Integer>`.
A page is considered repeatedly visited if its ID occurs at least twice.
Create another `LinkedList` containing each repeated ID only once.
The result must follow the order in which the repeated IDs first appeared.
Do not modify the original list.

Approach

Use a frequency map to count occurrences.
Then traverse the original list again:

- if frequency is greater than `1`;
- and the ID has not already been added to the result;
- add it.

Full Java Running Code

```
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.Scanner;

public class Main {

    public static LinkedList<Integer> findRepeatedPages(
            LinkedList<Integer> pages) {

        HashMap<Integer, Integer> frequency =
                new HashMap<>();

        for (int page : pages) {
            frequency.put(
                    page,
                    frequency.getOrDefault(page, 0) + 1
            );
        }

        LinkedList<Integer> result =
                new LinkedList<>();

        HashSet<Integer> added = new HashSet<>();

        for (int page : pages) {

            if (frequency.get(page) > 1 &&
                    !added.contains(page)) {

                result.add(page);
                added.add(page);
            }
        }

        return result;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        LinkedList<Integer> pages =
                new LinkedList<>();

        for (int i = 0; i < n; i++) {
            pages.add(sc.nextInt());
        }

        LinkedList<Integer> result =
                findRepeatedPages(pages);

        for (int page : result) {
            System.out.print(page + " ");
        }

        sc.close();
    }
}
```

Explanation

The `HashMap` stores how many times each page occurs.
For example:

```
5 → 3
7 → 1
9 → 2
```

means page `5` appeared three times and page `9` twice.
The second traversal preserves the original order, while `HashSet added` prevents the same repeated page from entering the result more than once.

Question 9 — Remove Duplicate Employee IDs

Problem Statement

A company receives a list of employee IDs from several departments.
Some IDs may appear multiple times.
Use a `HashSet<Integer>` to create a collection containing every distinct employee ID.
Display the unique IDs.

Approach

Insert every ID into a `HashSet`.
Since a `HashSet` does not retain duplicate values, repeated IDs are automatically ignored.

Full Java Running Code

```java
import java.util.HashSet;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        HashSet<Integer> employees =
                new HashSet<>();

        for (int i = 0; i < n; i++) {
            employees.add(sc.nextInt());
        }

        for (int id : employees) {
            System.out.print(id + " ");
        }

        sc.close();
    }
}
```

Explanation

The key operation is:

```
employees.add(id);
```

A `HashSet` stores only unique elements.
Therefore, if the same employee ID is inserted several times, only one copy remains.

Question 10 — Find Common Product Codes

Problem Statement

Two warehouses maintain collections of product IDs.
Find all product IDs that are present in **both** warehouses.
Each common ID should be printed only once.

Approach

1. Store the first warehouse's IDs in a `HashSet`.
2. Traverse the second warehouse.
3. Check whether each ID exists in the first set.
4. Add common values to another set.

Full Java Running Code

```
import java.util.HashSet;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        HashSet<Integer> warehouseA =
                new HashSet<>();

        for (int i = 0; i < n; i++) {
            warehouseA.add(sc.nextInt());
        }

        int m = sc.nextInt();

        HashSet<Integer> common =
                new HashSet<>();

        for (int i = 0; i < m; i++) {

            int id = sc.nextInt();

            if (warehouseA.contains(id)) {
                common.add(id);
            }
        }

        for (int id : common) {
            System.out.print(id + " ");
        }

        sc.close();
    }
}
```

Explanation

The first set provides fast membership checking:

```
warehouseA.contains(id)
```

If an ID exists there and also appears in the second warehouse, it is inserted into `common`.
Because `common` is also a `HashSet`, duplicates are automatically removed.

Question 11 — Find the Second Distinct Lowest Value

Problem Statement

For each test case, you are given a list of integers.
Find the **second smallest distinct value**.
If all elements are identical or there is only one distinct value, print:

```
0
```

Approach

1. Insert all values into a `HashSet` to remove duplicates.
2. Convert the set into an `ArrayList`.
3. Sort the list.
4. The element at index `1` is the second smallest distinct value.

Full Java Running Code

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.Scanner;

public class Main {

    public static int secondMinimum(int[] arr) {

        HashSet<Integer> unique =
                new HashSet<>();

        for (int value : arr) {
            unique.add(value);
        }

        if (unique.size() < 2) {
            return 0;
        }

        ArrayList<Integer> values =
                new ArrayList<>(unique);

        Collections.sort(values);

        return values.get(1);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int t = sc.nextInt();

        while (t-- > 0) {

            int n = sc.nextInt();

            int[] arr = new int[n];

            for (int i = 0; i < n; i++) {
                arr[i] = sc.nextInt();
            }

            System.out.println(secondMinimum(arr));
        }

        sc.close();
    }
}
```

Explanation

The `HashSet` first removes repeated values.
For:

```
20 5 15 5 10 20
```

the unique values are:

```
5 10 15 20
```

After sorting, index `1` gives `10`.
This builds directly on the distinct-value reasoning used in the collection-related material.

Question 12 — Count Product Frequencies

Problem Statement

An online store records product codes for every purchase.
Determine how many times each product code occurs.
Use a `HashMap<Integer, Integer>` to store the frequency of every product.
Display each product code together with its frequency.

Approach

For every product code:

```
map.put(code, map.getOrDefault(code, 0) + 1);
```

This either creates the first count or increases the existing count.

Full Java Running Code

```
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        HashMap<Integer, Integer> frequency =
                new HashMap<>();

        for (int i = 0; i < n; i++) {

            int code = sc.nextInt();

            frequency.put(
                    code,
                    frequency.getOrDefault(code, 0) + 1
            );
        }

        for (Map.Entry<Integer, Integer> entry :
                frequency.entrySet()) {

            System.out.println(
                    entry.getKey() + " " +
                    entry.getValue()
            );
        }

        sc.close();
    }
}
```

Explanation

The map stores data in:

```
key → frequency
```

For example:

```
101 → 3
205 → 2
310 → 1
```

`getOrDefault()` makes it easy to initialize a count to zero before increasing it.

Question 13 — Find the First Unique Character

Problem Statement

A messaging application receives a lowercase string.
Find the **first character that appears exactly once**.
Print its index.
If every character occurs more than once, print:

```
-1
```

Approach

Use a `HashMap<Character, Integer>` to count the frequency of every character.
Then traverse the string from left to right again.
The first character whose frequency is `1` gives the answer.

Full Java Running Code

```
import java.util.HashMap;
import java.util.Scanner;

public class Main {

    public static int firstUniqueIndex(String text) {

        HashMap<Character, Integer> frequency =
                new HashMap<>();

        for (char ch : text.toCharArray()) {
            frequency.put(
                    ch,
                    frequency.getOrDefault(ch, 0) + 1
            );
        }

        for (int i = 0; i < text.length(); i++) {

            if (frequency.get(text.charAt(i)) == 1) {
                return i;
            }
        }

        return -1;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        String text = sc.nextLine();

        System.out.println(
                firstUniqueIndex(text)
        );

        sc.close();
    }
}
```

Explanation

The first loop builds character frequencies.
The second loop checks characters in their original order.
For example:

```
swiss
```

has frequencies:

```
s → 3
w → 1
i → 1
```

The first unique character is `w`, so its index is returned.

Question 14 — Preserve Unique Notification Types

Problem Statement

A monitoring system receives notification types in the order they are generated.
For example:

```
EMAIL SMS EMAIL ALERT SMS PUSH
```

Create a collection that contains each notification type only once while preserving its **first appearance order**.
Use an appropriate Java collection.

Approach

Use `LinkedHashSet<String>`.
Unlike `HashSet`, `LinkedHashSet` maintains insertion order while still eliminating duplicates.

Full Java Running Code

```java
import java.util.LinkedHashSet;
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        LinkedHashSet<String> notifications =
                new LinkedHashSet<>();

        for (int i = 0; i < n; i++) {
            notifications.add(sc.next());
        }

        for (String type : notifications) {
            System.out.print(type + " ");
        }

        sc.close();
    }
}
```

Explanation

`LinkedHashSet` provides two useful properties:

- duplicate values are removed;
- insertion order is preserved.

So:

```
EMAIL SMS EMAIL ALERT SMS PUSH
```

becomes:

```
EMAIL SMS ALERT PUSH
```

This is useful when a problem requires both **uniqueness and original order**.

Question 15 — Analyze Repeated Student Activity

Problem Statement

A learning platform records the IDs of students who accessed a particular course.
You must determine:

1. how many distinct students accessed the course;
2. which student accessed it the maximum number of times;
3. the frequency of that student.

If multiple students have the same highest frequency, choose the student whose **first appearance occurred earlier**.
Use Java Collections.

Approach

Use two collections:

- `HashMap<Integer, Integer>` to store each student's frequency.
- `ArrayList<Integer>` to preserve the original access sequence.

First count every student.
Then traverse the original list and find the student with the highest frequency.
Because the traversal follows the original order, the first student encountered with the maximum frequency is retained.

Full Java Running Code

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Scanner;

public class Main {

    public static void analyze(
            ArrayList<Integer> accesses) {

        HashMap<Integer, Integer> frequency =
                new HashMap<>();

        for (int student : accesses) {

            frequency.put(
                    student,
                    frequency.getOrDefault(student, 0) + 1
            );
        }

        int distinctStudents = frequency.size();

        int bestStudent = accesses.get(0);
        int bestFrequency = frequency.get(bestStudent);

        for (int student : accesses) {

            int currentFrequency =
                    frequency.get(student);

            if (currentFrequency > bestFrequency) {

                bestFrequency = currentFrequency;
                bestStudent = student;
            }
        }

        System.out.println(distinctStudents);
        System.out.println(bestStudent);
        System.out.println(bestFrequency);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        ArrayList<Integer> accesses =
                new ArrayList<>();

        for (int i = 0; i < n; i++) {
            accesses.add(sc.nextInt());
        }

        analyze(accesses);

        sc.close();
    }
}
```

Explanation

The `HashMap` calculates how frequently each student appears.
For example:

```
12 15 12 18 15 12
```

produces:

```
12 → 3
15 → 2
18 → 1
```

So:

- distinct students = `3`
- most frequent student = `12`
- frequency = `3`

The original `ArrayList` is used when resolving ties because it preserves the access order.